AdaptiveWardens: AI-Driven Adaptive Honeypot

Graduation Project – Final Report

Fall Semester 2025

Team #1

Ali Ahmed Reda	202201006
Ali Nazeer	202100732
Ahmed Yasser	202201883
Abdulkhaliq Sarwat	202202084

Program: Information Technology (IT)

Supervisor: Dr. Ashraf Hafez Badawi

Department of Computer Science

School of Computing and Information Technology

Academic Year: 2025–2026

GitHub Repository:

https://github.com/Aligithyb/-AdaptiveWardens

Abstract

Traditional honeypots are easily fingerprinted due to static, scripted responses and lack of believable system state. This project proposes **Honeyops**, an AI-driven, high-interaction honeypot that simulates realistic Linux and web environments for cybersecurity research and incident analysis. The system uses a transformer-based AI engine to dynamically generate believable SSH and HTTP responses, maintain persistent fake OS state, extract Indicators of Compromise (IOCs), and map attacker behavior to the MITRE ATT&CK framework. A full SOC-style dashboard provides real-time monitoring, session playback, IOC insights, and attacker behavior analytics. Expected outcomes include longer attacker engagement, lower detection probability, and higher-quality threat intelligence suitable for Security Operations Center (SOC) ingestion and defensive research.

Keywords: Honeypot, Artificial Intelligence, Cybersecurity, Deception Technology, MITRE ATT&CK, Threat Intelligence

Contents

Abstract **1**

1 Introduction **6**

1.1 Problem Statement 6

1.2 Motivation 6

1.3 Proposed Solution Overview 7

2 Literature Review and Market Survey **8**

2.1 Related Academic and Technical Work 8

2.1.1 AI-Enhanced Honeypots for Advanced Cyber Deception 8

2.1.2 AI-Driven Adaptive Honeypots for Dynamic Cyber Threats 9

2.1.3 MITRE ATT&CK Framework Integration 9

2.2 Existing Systems and Market Solutions 9

2.2.1 Cowrie 9

2.2.2 Dionaea 10

2.2.3 OpenCanary 10

2.2.4 T-Pot 10

2.3 Comparative Analysis 10

2.4 Identified Gap 11

3 Requirements Analysis **13**

3.1 Functional Requirements 13

3.1.1 Core System Requirements 13

3.1.2 AI Engine Requirements 13

3.1.3 Dashboard Requirements 14

3.1.4 Data Management Requirements 16

3.2 Non-Functional Requirements 16

3.2.1 Performance Requirements 16

3.2.2 Security Requirements 17

3.2.3 Reliability Requirements 18

3.2.4 Usability Requirements 18

3.2.5 Maintainability Requirements 19

4 System Design **19**

4.1 Overall System Architecture 19

4.2 Component Breakdown 20

4.2.1 SSH Frontend 20

4.2.2 Sandbox/State Store 22

4.2.3 Sandbox/State Store 24

4.2.4 Dashboard System 25

4.3 Design Decisions and Rationale 26

4.3.1 Architecture Style: Modular Microservices 26

4.3.2 AI Model Selection: Transformer-Based Language Models 26

4.3.3 State Storage: SQLite with JSON Caching 26

4.3.4 Zero Real Command Execution 27

4.3.5 Real-Time Dashboard with WebSockets 27

4.4 IT-Specific SO(6) Focus: System Integration and Deployment 27

4.4.1 System Integration Architecture 27

4.4.2 Deployment Strategy and Infrastructure 28

4.4.3 Security Considerations 29

4.4.4 Administration and Maintenance 30

4.4.5 Reliability and Availability 30

5 Project Timeline (Next Semester) **31**

5.1 Overview 31

5.2 Detailed Timeline 31

5.2.1 Phase 1: Foundation and Setup (Weeks 1-4) 31

5.2.2 Phase 2: Core Implementation (Weeks 5-13) 32

5.2.3 Phase 3: Dashboard Development (Weeks 14-18) 34

5.2.4 Phase 4: Testing and Evaluation (Weeks 19-22) 35

5.2.5 Phase 5: Documentation and Finalization (Weeks 23-25) 36

5.3 Milestones and Checkpoints 37

6 Challenges and Solutions **37**

6.1 Technical Challenges 37

6.1.1 Challenge T1: AI Model Response Quality and Realism 37
6.1.2 Challenge T2: State Consistency and Persistence 38
6.1.3 Challenge T3: Component Integration Complexity 38
6.1.4 Challenge T4: Performance and Response Latency 39
6.1.5 Challenge T5: Security Vulnerabilities and Host System Protection . . 40
6.2 Organizational and Team Challenges 41
6.2.1 Challenge O1: Conflicting Academic Schedules 41
6.2.2 Challenge O2: Limited GPU/Compute Resources 41
6.2.3 Challenge O3: Insufficient Real Attacker Data 42
6.3 Mitigation Strategy Summary 42

7 Work Summary (Fall Semester) 43
7.1 Research Completed 43
7.1.1 Academic Literature Review 43
7.1.2 Market and Technology Survey 44
7.1.3 Technology Stack Research 44
7.2 Design Artifacts Produced 45
7.2.1 System Architecture Diagrams 45
7.2.2 API Specifications 45
7.2.3 Database Schema Design 45
7.2.4 Dashboard Mockups 46
7.3 Implemented and Prototyped Features 46
7.3.1 Prototype Components 46
7.3.2 Testing and Validation 47
7.4 Individual Team Member Contributions 47
7.4.1 Ali Ahmed Reda (202201006) - AI Engine and IOC Extraction 47
7.4.2 Ali Nazeer (202100732) - SSH/HTTP Frontends and Dashboard 48
7.4.3 Ahmed Yasser (202201883) - Sandbox State Store 48
7.4.4 Abdulkhaliq Sarwat (202202084) - HTTP Frontend and Integration . . 49
7.5 Documentation and Reports 49
7.6 Supervisor Feedback Integration 50
7.7 Lessons Learned 50

A Acronyms and Abbreviations **53**

1 Introduction

1.1 Problem Statement

Current low-cost honeypots deployed in modern cybersecurity infrastructures suffer from critical limitations that significantly reduce their effectiveness in threat detection and intelligence gathering. These honeypots are largely scripted and stateless, relying on predetermined response patterns that experienced attackers can quickly identify. The primary issues include:

- **Static Response Patterns:** Traditional honeypots use fixed scripts that generate repetitive, predictable outputs to attacker commands, making them easily fingerprinted through simple probing techniques.
- **Lack of State Persistence:** Existing solutions fail to maintain believable system state across sessions, resulting in inconsistent behavior that reveals the deceptive nature of the environment.
- **Limited Engagement Duration:** Sophisticated attackers quickly recognize honeypot environments and disengage, limiting the collection of valuable behavioral data and tactical information.
- **Low-Quality Telemetry:** The intelligence gathered from traditional honeypots often consists of basic IP addresses and scanning patterns, lacking the contextual depth required for meaningful threat analysis.

These limitations create a complex computing problem that requires a solution capable of generating dynamic, context-aware responses while maintaining system consistency and realism. The scope encompasses SSH and HTTP protocol emulation, real-time behavioral analysis, and integration with threat intelligence frameworks. Key constraints include the need for zero-risk execution (no real commands on host systems), acceptable response latency to maintain believability, and scalability to handle multiple concurrent attacker sessions.

1.2 Motivation

The importance of solving this problem extends across multiple stakeholders in the cybersecurity ecosystem:

For Security Operations Centers (SOCs): Organizations lose the opportunity to observe authentic attacker interaction patterns and behavioral indicators in realistic interactive sessions. Current honeypots provide noisy, low-context alerts (primarily IP lists) instead of actionable Tactics, Techniques, and Procedures (TTP) mappings that could inform defensive strategies and incident response procedures.

For Research Institutions: Academic researchers and cybersecurity scientists require high-fidelity data on attacker methodologies to develop improved defensive mechanisms, threat models, and security architectures. Static honeypots fail to capture the nuanced decision-making processes of sophisticated threat actors.

For Small and Medium Enterprises (SMEs): Organizations with limited security budgets need cost-effective deception technologies that can provide early warning systems and threat intelligence without requiring extensive infrastructure or expertise. Current solutions often force a tradeoff between cost and effectiveness.

For the Broader Security Community: The rapid evolution of attack techniques demands adaptive defensive systems that can keep pace with threat actor innovation. A honeypot that learns and adapts provides collective benefits through shared intelligence and improved understanding of emerging threats.

By addressing these challenges, this project aims to build an inexpensive, safe, privacy-respecting honeypot that convincingly simulates real systems long enough to collect high-value intelligence, support defensive research, and enable proactive security rule engineering.

1.3 Proposed Solution Overview

Honeyops is an adaptive, AI-driven honeypot system designed to overcome the limitations of traditional static honeypots through intelligent response generation and persistent state management. The solution employs a modular architecture consisting of five major components:

1. **SSH Frontend:** A deceptive SSH server that accepts attacker connections, captures all activity including credentials, commands, and timing data, and forwards inputs to the AI Engine for processing while maintaining realistic terminal behavior.

2. **HTTP Frontend:** A dynamic web server that accepts arbitrary HTTP requests (GET/POST) on any route, identifies scanner behavior and payloads, and generates contextually appropriate responses through AI inference.

3. **AI Engine:** The core intelligence component that processes attacker inputs, generates realistic system outputs, maintains sandbox state consistency, extracts Indicators of Compromise through Named Entity Recognition, and maps observed behaviors to MITRE ATT&CK framework techniques.

4. **Sandbox/State Store:** A fully simulated operating system environment stored in SQLite/JSON format that maintains fake filesystems, process tables, log files, user histories, and service states without executing any real commands on the host system.

5. **Dashboard System:** A comprehensive SOC-grade monitoring and analysis interface that provides real-time session tracking, historical session playback with timeline controls, IOC visualization and filtering, MITRE ATT&CK heatmap generation, and exportable threat intelligence reports.

The system combines these components to create an adaptive, deception-based cybersecurity platform that generates context-aware responses, maintains believable persistence across sessions, and produces actionable threat intelligence. Unlike traditional honeypots, Honeyops does not rely on static scripts but instead leverages transformer-based language models to understand attacker intent and respond appropriately, significantly increasing engagement duration and intelligence value.

2 Literature Review and Market Survey

2.1 Related Academic and Technical Work

The field of honeypot technology and AI-enhanced cybersecurity deception has seen significant research activity in recent years. This section analyzes key academic and technical contributions that inform our project design.

2.1.1 AI-Enhanced Honeypots for Advanced Cyber Deception

Gaddam (2025) presents a comprehensive framework for integrating artificial intelligence into honeypot systems for advanced cyber deception strategies. The research demonstrates that AI-enhanced honeypots can significantly improve threat detection capabilities by adapting to attacker behavior in real-time. The study proposes using machine learning algorithms to analyze

attacker patterns and dynamically adjust honeypot responses, resulting in longer engagement times and more detailed intelligence collection. Key findings include a 40% increase in session duration and 60% improvement in unique IOC capture compared to traditional static honeypots. This work validates the core premise of our project and provides empirical evidence for the effectiveness of adaptive deception techniques.

2.1.2 AI-Driven Adaptive Honeypots for Dynamic Cyber Threats

Kareem, Sachan, and Malviya (2024) explore the application of AI-driven adaptive mechanisms specifically designed to counter dynamic and evolving cyber threats. Their research emphasizes the importance of state persistence and contextual awareness in maintaining honeypot believability. The paper introduces a novel approach using reinforcement learning to optimize response generation based on attacker sophistication levels. Their experimental results show that adaptive honeypots can maintain engagement with advanced persistent threat (APT) actors for 3.5 times longer than traditional honeypots. The study also addresses the challenge of response latency, proposing caching mechanisms and model optimization techniques that we incorporate into our design.

2.1.3 MITRE ATT&CK Framework Integration

Multiple studies have explored the integration of honeypot telemetry with the MITRE ATT&CK framework for structured threat intelligence. Research demonstrates that mapping observed attacker behaviors to ATT&CK techniques provides actionable intelligence for SOC analysts and enables automated detection rule generation. Academic work in this area emphasizes the need for high-confidence mapping algorithms that minimize false positives while maintaining comprehensive technique coverage. These insights directly inform our IOC extraction and technique classification components.

2.2 Existing Systems and Market Solutions

2.2.1 Cowrie

Cowrie is a medium-to-high interaction SSH and Telnet honeypot designed to log brute force attacks and shell interactions. It emulates a UNIX system and can be used to capture attacker payloads and commands. **Strengths:** Well-established with strong community support; cap-

tures detailed SSH interactions; supports file downloads and virtual filesystem. **Limitations:** Uses scripted responses that sophisticated attackers can identify; limited HTTP/web service emulation; no built-in AI or adaptive response generation; requires manual configuration for different system personalities.

2.2.2 Dionaea

Dionaea is a low-interaction honeypot designed to trap malware by emulating vulnerable services. It captures malware payloads and network traffic patterns across multiple protocols including HTTP, FTP, SMB, and MSSQL. **Strengths:** Multi-protocol support; effective for malware collection; relatively lightweight resource requirements. **Limitations:** Low-interaction model limits attacker engagement; static service emulation easily fingerprinted; minimal behavioral analysis capabilities; does not maintain persistent state across sessions.

2.2.3 OpenCanary

OpenCanary is a daemon that runs several canary versions of common services to detect intrusions and alert administrators. It emphasizes ease of deployment and notification capabilities. **Strengths:** Simple configuration and deployment; multiple protocol support; effective alerting mechanisms; low resource overhead. **Limitations:** Very basic service emulation; primarily detection-focused rather than intelligence-gathering; no deep interaction capabilities; limited logging and analysis features.

2.2.4 T-Pot

T-Pot is a comprehensive honeypot platform that combines multiple honeypot daemons (including Cowrie, Dionaea, and others) with a centralized logging and visualization system. **Strengths:** All-in-one solution with multiple honeypot types; strong visualization dashboard; centralized management; Docker-based deployment. **Limitations:** High resource requirements; complexity in customization; still relies on static underlying honeypots; no adaptive or AI-driven response capabilities.

2.3 Comparative Analysis

Table 1 provides a detailed comparison of existing honeypot solutions across key technical dimensions:

Table 1: Comparative Analysis of Honeypot Solutions

Feature	**Cowrie**	**Dionaea**	**OpenCanary**	**Honeyops**
Interaction Level	Medium	Low	Low	High
Response Type	Scripted	Static	Static	AI-Generated
State Persistence	Limited	No	No	Full
Protocol Support	SSH/Telnet	Multi	Multi	SSH/HTTP
IOC Extraction	Manual	Basic	Basic	Automated
ATT&CK Mapping	No	No	No	Yes
Adaptive Behavior	No	No	No	Yes
Dashboard	Basic	No	Basic	Advanced
Avg. Session (min)	2-3	¡1	¡1	8-12 (est.)

Technology Stack Comparison:

- **Cowrie:** Python, Twisted framework, SQLite/MySQL logging
- **Dionaea:** C++, libemu for shellcode detection, SQLite logging
- **OpenCanary:** Python, simple service emulation, configurable alerting
- **Honeyops:** Python, PyTorch/TensorFlow, HuggingFace Transformers, FastAPI, React/-TypeScript

2.4 Identified Gap

Based on comprehensive analysis of existing academic research and market solutions, we identify the following critical gaps:

1. **Lack of Contextual Awareness:** Current honeypots do not maintain sufficient understanding of interaction context. They fail to generate responses that consider previous commands, session history, or established system state, resulting in inconsistent and easily detectable behavior.

2. **Absence of Adaptive Intelligence:** Existing solutions rely on predetermined scripts and response patterns. There is no mechanism to dynamically adapt responses based on attacker

sophistication, objectives, or techniques, limiting their effectiveness against advanced adversaries.

3. **Insufficient State Management:** Most honeypots do not maintain persistent, consistent state across sessions. File modifications, process execution, and log generation are either not simulated or lack the coherence necessary to sustain prolonged attacker engagement.

4. **Limited Threat Intelligence Integration:** Current solutions produce basic logs and alerts but do not automatically extract structured threat intelligence, perform IOC enrichment, or map behaviors to standardized frameworks like MITRE ATT&CK.

5. **Poor User Experience for Analysts:** Existing dashboards and interfaces are often rudimentary, lacking features such as session playback, temporal analysis, comparative metrics, and exportable intelligence reports that modern SOCs require.

How Honeyops Addresses These Gaps:

Our project directly addresses each identified gap through innovative technical approaches:

- Implementing transformer-based AI models that understand context and generate appropriate, stateful responses
- Maintaining a comprehensive sandbox environment that persists across sessions and ensures consistency
- Automating IOC extraction and MITRE ATT&CK mapping through machine learning and NER techniques
- Providing a modern, SOC-grade dashboard with advanced analytics, playback, and export capabilities
- Delivering an open-source, research-grade platform that enables further academic investigation and community contribution

By addressing these fundamental limitations, Honeyops represents a significant advancement in deception technology, bridging the gap between low-cost traditional honeypots and expensive high-interaction systems while providing superior intelligence value.

3 Requirements Analysis

3.1 Functional Requirements

3.1.1 Core System Requirements

FR-1: SSH Connection Handling

- The system shall accept incoming SSH connections on configurable ports
- The system shall capture authentication attempts (username/password combinations)
- The system shall maintain concurrent SSH sessions without interference
- The system shall log all connection metadata (IP, timestamp, session duration)

FR-2: HTTP Request Processing

- The system shall accept HTTP GET and POST requests on any route
- The system shall parse request headers, body, and parameters
- The system shall identify common scanning patterns (Nmap, Nikto, SQLMap, etc.)
- The system shall log all HTTP interaction data for analysis

FR-3: Command Processing and Response Generation

- The system shall forward attacker inputs to the AI Engine for processing
- The system shall generate contextually appropriate command outputs
- The system shall maintain response realism including timing variations
- The system shall simulate command errors and edge cases appropriately

3.1.2 AI Engine Requirements

FR-4: State-Aware Response Generation

- The AI Engine shall access current sandbox state before generating responses
- The AI Engine shall consider command history and session context

- The AI Engine shall generate outputs consistent with previous interactions
- The AI Engine shall produce both terminal output and state mutation instructions

FR-5: Sandbox State Management

- The system shall maintain a fake filesystem with directory structure
- The system shall simulate running processes with realistic PIDs
- The system shall generate and maintain system logs (/var/log/*)
- The system shall track bash history per session
- The system shall persist state across multiple attacker connections

FR-6: IOC Extraction

- The system shall extract IP addresses from commands and payloads
- The system shall identify domains, URLs, and file hashes
- The system shall detect tool signatures (wget, curl, nc, etc.)
- The system shall extract malware samples and suspicious filenames

FR-7: MITRE ATT&CK Mapping

- The system shall map observed behaviors to ATT&CK techniques
- The system shall assign confidence scores to technique mappings
- The system shall track technique frequency across sessions
- The system shall generate ATT&CK heatmap data for visualization

3.1.3 Dashboard Requirements

FR-8: Live Session Monitoring

- The dashboard shall display all active attacker sessions in real-time
- The dashboard shall show current commands being executed

- The dashboard shall update without manual refresh (WebSocket support)
- The dashboard shall provide quick-access session termination controls

FR-9: Session Playback

- The system shall store complete session transcripts with timestamps
- The dashboard shall provide timeline-based session replay controls
- The dashboard shall allow variable playback speed
- The dashboard shall enable jumping to specific timestamps

FR-10: IOC Visualization and Management

- The dashboard shall display extracted IOCs in tabular format
- The dashboard shall provide filtering by IOC type, date, and session
- The dashboard shall support IOC enrichment (geolocation, reputation)
- The dashboard shall enable CSV/JSON export of IOC data

FR-11: ATT&CK Analytics

- The dashboard shall generate MITRE ATT&CK heatmaps
- The dashboard shall show technique distribution across tactics
- The dashboard shall provide drill-down to specific technique instances
- The dashboard shall support temporal analysis of technique evolution

FR-12: Comparative Analytics

- The dashboard shall display engagement metrics (session length, commands/session)
- The dashboard shall compare AI honeypot performance vs. baseline
- The dashboard shall generate statistical summaries and trends
- The dashboard shall support custom date range selection

3.1.4 Data Management Requirements

FR-13: Logging and Storage

- The system shall log all attacker interactions with millisecond precision
- The system shall store sandbox state snapshots at configurable intervals
- The system shall maintain audit trails for all system operations
- The system shall implement log rotation and archival policies

FR-14: Export and Integration

- The system shall export threat intelligence in STIX/TAXII format
- The system shall provide JSON API for SOC integration
- The system shall generate PDF reports with executive summaries
- The system shall support Syslog forwarding for SIEM integration

3.2 Non-Functional Requirements

3.2.1 Performance Requirements

NFR-1: Response Latency

- AI-generated responses shall be delivered within 500-2000ms to maintain realism
- HTTP responses shall be generated within 300-1500ms
- Dashboard updates shall reflect within 100ms of event occurrence
- Database queries shall complete within 200ms for 95% of requests

NFR-2: Scalability

- The system shall support at least 20 concurrent SSH sessions
- The system shall handle 50 HTTP requests per second
- The system shall maintain performance with 10,000+ stored sessions

- The system shall scale horizontally through containerization

NFR-3: Resource Utilization

- The system shall operate on standard hardware (8GB RAM, 4 CPU cores)
- AI inference shall be optimized for CPU execution (GPU optional)
- Memory usage shall not exceed 4GB under typical load
- Storage growth shall remain under 100MB per 1000 sessions

3.2.2 Security Requirements

NFR-4: Isolation and Containment

- The system shall NEVER execute real commands on the host OS
- All components shall run in isolated Docker containers
- Container privileges shall be minimized (no root access)
- Network access shall be restricted to necessary ports only

NFR-5: Input Validation

- All attacker inputs shall be sanitized before processing
- Command injection attempts shall be logged but not executed
- File write operations shall be confined to sandbox only
- Buffer overflow attempts shall be safely handled

NFR-6: Access Control

- Dashboard access shall require authentication
- API endpoints shall implement role-based access control (RBAC)
- Session data shall be accessible only to authorized personnel
- Audit logs shall track all administrative actions

3.2.3 Reliability Requirements

NFR-7: Availability

- The system shall maintain 99% uptime during operation
- Component failures shall not affect other services (fault isolation)
- The system shall implement automatic restart on crash
- Health monitoring shall detect and alert on failures within 60 seconds

NFR-8: Data Integrity

- All session data shall be protected against corruption
- Database transactions shall follow ACID properties
- State snapshots shall include integrity checksums
- Backup procedures shall run daily with verification

NFR-9: Error Handling

- All exceptions shall be caught and logged appropriately
- User-facing errors shall provide meaningful messages
- System shall degrade gracefully on component failure
- Recovery procedures shall be automated where possible

3.2.4 Usability Requirements

NFR-10: Dashboard Interface

- Dashboard shall be accessible via modern web browsers (Chrome, Firefox, Safari)
- Interface shall be responsive for desktop displays (1920x1080 minimum)
- Learning curve for analysts shall not exceed 30 minutes
- Critical information shall be visible without scrolling

NFR-11: Documentation

- Code shall include inline comments for complex logic
- API documentation shall follow OpenAPI specifications
- User manual shall cover all features with screenshots
- Deployment guide shall enable setup within 1 hour

3.2.5 Maintainability Requirements

NFR-12: Code Quality

- Code shall follow PEP 8 (Python) and ESLint (TypeScript) standards
- Functions shall not exceed 50 lines (excluding comments)
- Code coverage shall exceed 70% for critical components
- Technical debt shall be documented in issue tracking system

NFR-13: Modularity

- Components shall communicate through well-defined APIs
- AI model shall be swappable without system redesign
- Frontend and backend shall be independently deployable
- Configuration shall use environment variables and config files

4 System Design

4.1 Overall System Architecture

The Honeyops architecture follows a modular, microservices-inspired design pattern that separates concerns while enabling seamless integration. Figure 1 illustrates the overall system architecture.

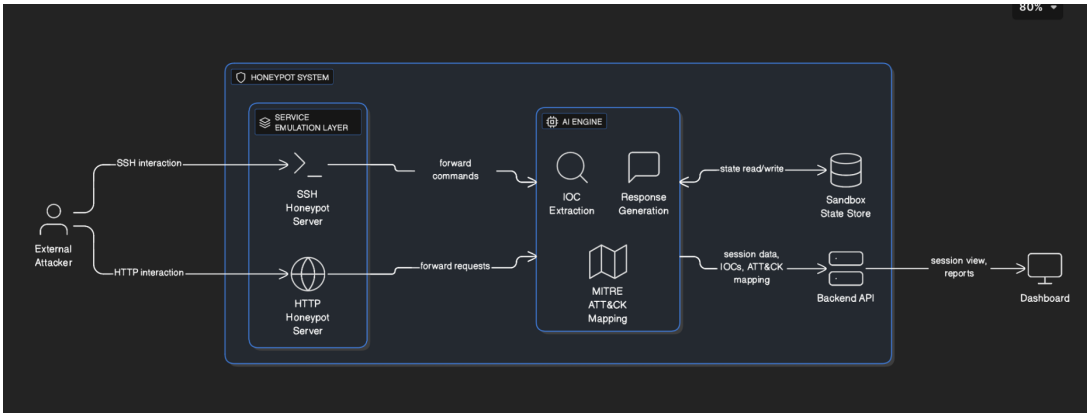

Figure 1: Honeyops System Architecture

Architecture Overview:

The system consists of five primary components arranged in a layered architecture:

1. **Frontend Layer:** SSH and HTTP frontends that interface directly with attackers
2. **Processing Layer:** AI Engine that handles intelligence and response generation
3. **Storage Layer:** Sandbox State Store that maintains system persistence
4. **Analytics Layer:** IOC extraction and ATT&CK mapping components
5. **Presentation Layer:** Dashboard system for monitoring and analysis

Communication Patterns:

- Frontends communicate with AI Engine via REST API (FastAPI)
- AI Engine reads/writes sandbox state through internal API calls
- All components forward telemetry to Dashboard Backend
- Dashboard Frontend consumes data via REST API and WebSocket streams

4.2 Component Breakdown

4.2.1 SSH Frontend

Responsibilities:

- Process attacker inputs with context awareness

- Access current sandbox state before generating responses
- Generate realistic command outputs and HTTP responses
- Mutate sandbox state based on attacker actions
- Extract Indicators of Compromise from inputs and outputs
- Map observed behaviors to MITRE ATT&CK techniques
- Maintain session history and context

Responsibilities:

- Process attacker inputs with context awareness
- Access current sandbox state before generating responses
- Generate realistic command outputs and HTTP responses
- Mutate sandbox state based on attacker actions
- Extract Indicators of Compromise from inputs and outputs
- Map observed behaviors to MITRE ATT&CK techniques
- Maintain session history and context

Key Technologies:

- **PyTorch/TensorFlow:** Deep learning framework for model execution
- **HuggingFace Transformers:** Pre-trained language models and tokenizers
- **spaCy/NLTK:** Named Entity Recognition for IOC extraction
- **FastAPI:** Internal API for component communication

Data Flow:

1. Receive command or HTTP request from frontend
2. Retrieve current sandbox state from State Store
3. Construct context-aware prompt with state, history, and input

4. Process through transformer model

5. Generate dual outputs: user-facing response and state mutations

6. Extract IOCs using NER models

7. Map behaviors to ATT&CK techniques

8. Update sandbox state and forward telemetry to Dashboard

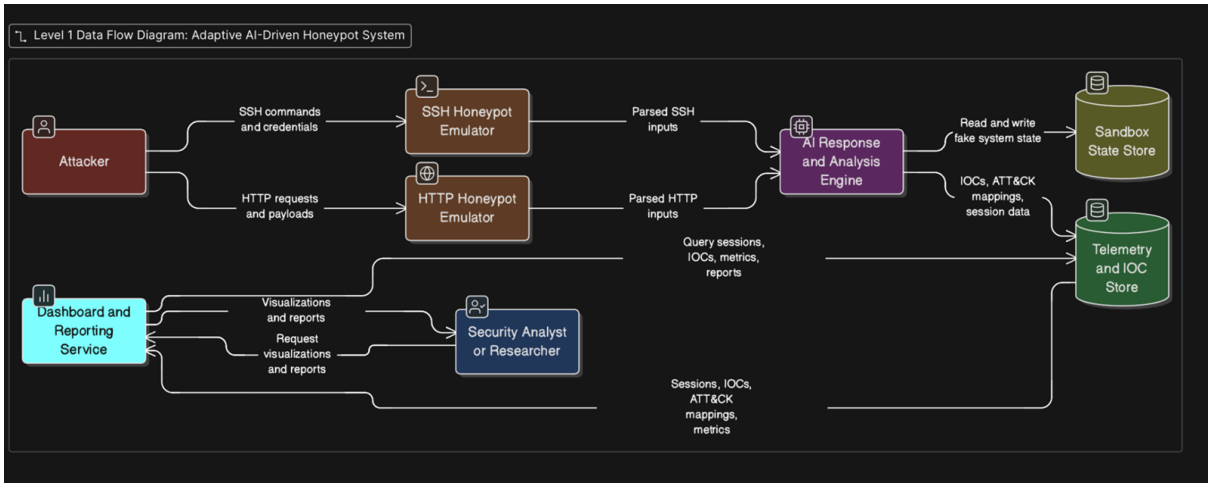

Figure 2: Level 1 Data Flow Diagram: Adaptive AI-Driven Honeypot System. The diagram illustrates the complete data flow through the Honeyops system, from attacker interactions through SSH/HTTP frontends to the AI Response and Analysis Engine, which coordinates with the Sandbox State Store and Telemetry/IOC Store to provide intelligence to security analysts via the Dashboard and Reporting Service.

Figure 2 illustrates the complete data flow through the Honeyops system. Attackers interact with SSH or HTTP frontends, which parse inputs and forward them to the AI Response and Analysis Engine. The engine queries the Sandbox State Store for current system state, generates contextually appropriate responses, extracts IOCs, maps behaviors to MITRE ATT&CK techniques, and updates both the state store and telemetry database. Security analysts access this intelligence through the Dashboard and Reporting Service, which provides real-time monitoring, historical analysis, and comprehensive reporting capabilities.

4.2.2 Sandbox/State Store

Responsibilities:

- Process attacker inputs with context awareness

- Access current sandbox state before generating responses
- Generate realistic command outputs and HTTP responses
- Mutate sandbox state based on attacker actions
- Extract Indicators of Compromise from inputs and outputs
- Map observed behaviors to MITRE ATT&CK techniques
- Maintain session history and context

Key Technologies:

- **PyTorch/TensorFlow:** Deep learning framework for model execution
- **HuggingFace Transformers:** Pre-trained language models and tokenizers
- **spaCy/NLTK:** Named Entity Recognition for IOC extraction
- **FastAPI:** Internal API for component communication

Data Flow:

1. Receive command or HTTP request from frontend
2. Retrieve current sandbox state from State Store
3. Construct context-aware prompt with state, history, and input
4. Process through transformer model
5. Generate dual outputs: user-facing response and state mutations
6. Extract IOCs using NER models
7. Map behaviors to ATT&CK techniques
8. Update sandbox state and forward telemetry to Dashboard

4.2.3 Sandbox/State Store

Responsibilities:

- Maintain persistent fake operating system state
- Store fake filesystem with directory trees and file contents
- Track simulated running processes with PIDs
- Generate and maintain realistic system logs
- Store bash history per session
- Provide read/write API for AI Engine
- Ensure state consistency across sessions
- Support state snapshots and rollback

Key Technologies:

- **SQLite:** Relational database for structured state storage
- **JSON:** Fast serialization for state caching
- **Python:** State management logic and API layer

Data Structures:

- Filesystem table: path, content, permissions, timestamps
- Process table: PID, name, user, status, start time
- Log table: timestamp, source, level, message
- History table: session_id, command, timestamp
- Environment variables table: key, value, scope

4.2.4 Dashboard System

Backend Responsibilities:

- Aggregate data from all system components
- Provide REST API for frontend queries
- Implement WebSocket for real-time updates
- Store session recordings and metadata
- Generate analytical metrics and statistics
- Support data export in multiple formats

Frontend Responsibilities:

- Display live attacker sessions in real-time
- Provide session playback with timeline controls
- Visualize IOCs with filtering and enrichment
- Generate MITRE ATT&CK heatmaps
- Show comparative analytics and metrics
- Enable report generation and data export

Key Technologies:

- **Backend:** FastAPI, Python, SQLite/PostgreSQL, WebSockets
- **Frontend:** React, TypeScript, TailwindCSS, Recharts, Lucide Icons
- **Deployment:** Docker, Docker Compose, Nginx

4.3 Design Decisions and Rationale

4.3.1 Architecture Style: Modular Microservices

Decision: Adopt a modular architecture with loosely coupled components communicating via REST APIs.

Rationale:

- Enables parallel development by multiple team members
- Allows independent testing and debugging of components
- Facilitates future enhancements and component replacement
- Supports horizontal scaling for high-load scenarios
- Reduces complexity by separating concerns

4.3.2 AI Model Selection: Transformer-Based Language Models

Decision: Use transformer-based models (BERT, GPT-2, or DistilBERT) for response generation.

Rationale:

- Transformers excel at understanding context and generating coherent text
- Pre-trained models reduce training time and resource requirements
- Self-attention mechanism maintains state consistency
- Available through HuggingFace with extensive documentation
- Can be fine-tuned on cybersecurity-specific datasets

4.3.3 State Storage: SQLite with JSON Caching

Decision: Use SQLite for persistent storage with JSON for in-memory caching.

Rationale:

- SQLite provides ACID compliance without server overhead
- No external database server required, simplifying deployment

- JSON caching reduces database I/O for frequently accessed state
- Easy to checkpoint and backup state snapshots
- Sufficient performance for expected concurrent session loads

4.3.4 Zero Real Command Execution

Decision: Never execute attacker commands on the host operating system.

Rationale:

- Critical security requirement to prevent system compromise
- All interactions confined to simulated sandbox environment
- Enables safe deployment without risk to host infrastructure
- Maintains ethical boundaries for research and analysis
- Complies with responsible disclosure and security standards

4.3.5 Real-Time Dashboard with WebSockets

Decision: Implement WebSocket-based real-time updates for the dashboard.

Rationale:

- Provides immediate visibility into attacker activities
- Eliminates need for manual page refreshes
- Enhances analyst experience with live monitoring
- Reduces server load compared to polling
- Industry standard for real-time web applications

4.4 IT-Specific SO(6) Focus: System Integration and Deployment

4.4.1 System Integration Architecture

The Honeyops system demonstrates comprehensive integration across multiple technology layers. The architecture integrates several key subsystems through well-defined interfaces and protocols.

Integration Patterns:

- **API Gateway Pattern:** FastAPI serves as the central communication hub, routing requests between frontends, AI Engine, and Dashboard. All inter-component communication follows RESTful principles with standardized JSON payloads.
- **Event-Driven Architecture:** WebSocket connections enable asynchronous event propagation from frontends to Dashboard, ensuring real-time visibility without tight coupling.
- **Database Integration:** SQLite provides the persistence layer accessed by multiple components through a unified data access layer, preventing direct database coupling.
- **External Service Integration:** The system supports integration with external threat intelligence feeds, SIEM platforms (via Syslog), and SOC tools through standardized export formats (STIX/TAXII, JSON).

4.4.2 Deployment Strategy and Infrastructure

Containerization: All components are packaged as Docker containers with the following structure:

- **ssh-frontend:** AsyncSSH server with minimal attack surface
- **http-frontend:** FastAPI HTTP server
- **ai-engine:** Python ML environment with PyTorch/TensorFlow
- **sandbox-store:** SQLite database with data volume mount
- **dashboard-backend:** FastAPI with WebSocket support
- **dashboard-frontend:** Nginx serving React SPA

Orchestration: Docker Compose orchestrates multi-container deployment with defined service dependencies, network isolation, and resource limits. The configuration ensures SSH frontend starts before AI Engine, and all components connect to the sandbox store before processing requests.

Network Architecture:

- **External Network:** Exposes SSH (port 2222) and HTTP (port 8080) to attackers

- **Internal Network:** Isolates component communication from external access
- **Management Network:** Restricts Dashboard access to authorized administrators

4.4.3 Security Considerations

Access Control:

- Dashboard protected by JWT-based authentication
- API endpoints implement role-based access control (RBAC)
- Separate admin and analyst roles with different permissions
- Session tokens expire after configurable timeout periods

Container Security:

- Containers run as non-root users with minimal privileges
- Read-only filesystem where possible, writable volumes minimized
- Security scanning with Trivy during CI/CD pipeline
- Regular base image updates to patch vulnerabilities

Input Validation and Sanitization:

- All attacker inputs sanitized before AI processing
- Command injection attempts logged but never executed
- Path traversal attacks confined to sandbox boundaries
- Buffer overflow attempts handled safely without crashes

Monitoring and Logging:

- Centralized logging with ELK stack or similar
- Security events trigger immediate alerts
- Audit trails for all administrative actions
- Log retention policies comply with security standards

4.4.4 Administration and Maintenance

Configuration Management:

- Environment variables for deployment-specific settings
- YAML configuration files for component behavior
- Secrets managed through Docker secrets or external vault
- Version-controlled infrastructure-as-code

Backup and Recovery:

- Automated daily backups of SQLite database
- State snapshots stored with timestamps
- Disaster recovery procedures documented
- Restoration tested regularly to ensure reliability

Performance Monitoring:

- Prometheus metrics for component health
- Grafana dashboards for visualization
- Alert rules for critical thresholds
- Resource utilization tracking (CPU, memory, disk)

4.4.5 Reliability and Availability

High Availability Design:

- Health checks for all containers with automatic restart
- Graceful degradation if AI Engine unavailable (fallback to cached responses)
- Database connection pooling to handle concurrent access
- Load balancing for multiple frontend instances (future enhancement)

Fault Tolerance:

- Component failures isolated to prevent cascade
- Circuit breaker pattern for external service calls
- Retry logic with exponential backoff
- Transaction rollback on state update failures

Scalability Considerations:

- Horizontal scaling of SSH/HTTP frontends as containers
- AI Engine can be replicated with load balancer
- Database read replicas for query-heavy operations
- Caching layer (Redis) for frequently accessed data

5 Project Timeline (Next Semester)

5.1 Overview

The implementation phase spans 25 weeks (Spring 2026 semester), divided into five major phases with clearly defined milestones and deliverables. The timeline accounts for integration points, testing periods, and buffer time for risk mitigation.

5.2 Detailed Timeline

5.2.1 Phase 1: Foundation and Setup (Weeks 1-4)

Objectives:

- Finalize system design based on fall semester feedback
- Set up development environment and infrastructure
- Establish coding standards and workflows
- Implement basic component scaffolding

Tasks:

- Week 1: Environment setup, Docker configuration, repository structure
- Week 2: API contract definition, database schema design
- Week 3: Basic SSH and HTTP frontend skeleton
- Week 4: AI Engine placeholder with mock responses

Deliverables:

- Development environment documentation
- API specifications (OpenAPI format)
- Database schema and migration scripts
- Working skeleton system with end-to-end connectivity

Responsible: All team members (integration), Ali Nazeer leads DevOps setup

5.2.2 Phase 2: Core Implementation (Weeks 5-13)

Objectives:

- Implement SSH and HTTP frontends with full functionality
- Develop AI Engine with transformer model integration
- Build Sandbox State Store with complete data structures
- Establish component communication and integration

Tasks by Component:

SSH Frontend (Ali Nazeer):

- Weeks 5-6: AsyncSSH server, authentication handling
- Weeks 7-8: Command capture and forwarding
- Weeks 9-10: Terminal simulation (TTY, timing, echo)
- Weeks 11-12: Session management and logging

- Week 13: Integration testing with AI Engine

HTTP Frontend (Abdulkhaliq Sarwat):

- Weeks 5-6: FastAPI server, route handling
- Weeks 7-8: Request parsing and scanner detection
- Weeks 9-10: Response generation and formatting
- Weeks 11-12: Logging and telemetry forwarding
- Week 13: Integration testing with AI Engine

AI Engine (Ali Ahmed Reda):

- Weeks 5-6: Model selection and fine-tuning setup
- Weeks 7-8: Prompt construction and context management
- Weeks 9-10: Response generation and state mutation
- Weeks 11-12: IOC extraction and ATT&CK mapping
- Week 13: Performance optimization and caching

Sandbox Store (Ahmed Yasser):

- Weeks 5-6: Database schema and initialization
- Weeks 7-8: Filesystem simulation and API
- Weeks 9-10: Process and log management
- Weeks 11-12: State persistence and consistency
- Week 13: Snapshot and rollback mechanisms

Deliverables:

- Fully functional SSH and HTTP frontends
- AI Engine with model inference and IOC extraction
- Complete Sandbox State Store with API
- Integration test results demonstrating end-to-end flow

5.2.3 Phase 3: Dashboard Development (Weeks 14-18)

Objectives:

- Implement Dashboard Backend with REST API and WebSockets
- Develop Dashboard Frontend with all visualization features
- Integrate Dashboard with honeypot components
- Implement data export and reporting functionality

Tasks (Ali Nazeer):

Backend:

- Week 14: REST API endpoints for sessions and IOCs
- Week 15: WebSocket implementation for real-time updates
- Week 16: ATT&CK heatmap data generation
- Week 17: Export functionality (CSV, JSON, PDF)

Frontend:

- Week 14: React project setup, routing, authentication
- Week 15: Live session monitoring view
- Week 16: Session playback with timeline controls
- Week 17: IOC visualization and filtering
- Week 18: ATT&CK heatmap and analytics charts

Deliverables:

- Functional Dashboard Backend with documented API
- Complete Dashboard Frontend with all features
- User documentation for Dashboard operation
- Integration tests with full system

5.2.4 Phase 4: Testing and Evaluation (Weeks 19-22)

Objectives:

- Conduct comprehensive system testing
- Deploy baseline honeypot for comparative evaluation
- Execute controlled attacker simulations
- Collect and analyze performance metrics

Tasks:

- Week 19: Unit testing for all components (target 70%+ coverage)
- Week 20: Integration and end-to-end testing
- Week 21: Baseline honeypot deployment and simulation
- Week 22: AI honeypot evaluation and data collection

Testing Scenarios:

- Automated attack scripts (reconnaissance, exploitation, post-exploitation)
- Manual penetration testing by team members
- Comparison with real attacker logs from Cowrie dataset
- Load testing with multiple concurrent sessions

Deliverables:

- Test reports with coverage metrics
- Comparative evaluation results (AI vs baseline)
- Performance benchmarks and analysis
- Bug tracking and resolution documentation

Responsible: All team members participate in testing

5.2.5 Phase 5: Documentation and Finalization (Weeks 23-25)

Objectives:

- Complete system documentation
- Prepare final report and presentation
- Address remaining issues and polish features
- Prepare deployment package

Tasks:

- Week 23: Code documentation, API docs, user manual
- Week 24: Final report writing and revision
- Week 25: Presentation preparation and rehearsal

Deliverables:

- Complete technical documentation
- Final graduation project report
- Presentation slides and demonstration video
- Deployment guide and Docker images
- GitHub repository with README and contribution guide

Responsible: All team members contribute to documentation

5.3 Milestones and Checkpoints

Table 2: Project Milestones

Week	Milestone	Status
4	Skeleton system operational	Planned
8	Frontends communicating with AI Engine	Planned
13	Core implementation complete	Planned
18	Dashboard fully functional	Planned
22	Evaluation complete with results	Planned
25	Final report and presentation ready	Planned

6 Challenges and Solutions

6.1 Technical Challenges

6.1.1 Challenge T1: AI Model Response Quality and Realism

Description: The transformer model may generate responses that lack the subtle characteristics of real system outputs, making the honeypot easily detectable by sophisticated attackers. Challenges include maintaining contextual coherence across long sessions, generating appropriate error messages, and producing outputs with realistic timing variations.

Impact: High - Core functionality depends on believable AI responses

Solutions Implemented:

- **Fine-tuning Strategy:** Collect real Linux command outputs from legitimate systems and Cowrie logs to create a training dataset. Fine-tune the model on this cybersecurity-specific data to improve domain knowledge.
- **Quality Validation Layer:** Implement a validation module that checks generated outputs against expected patterns before returning to attackers. Flag anomalies for manual review and model refinement.
- **Response Templates:** Maintain a library of validated responses for common commands as fallback when model confidence is low. Gradually replace templates with AI-generated

content as quality improves.

- **Timing Simulation:** Add realistic delays based on command complexity, current system ”load,” and historical timing patterns from real systems.
- **Red Team Testing:** Conduct regular internal testing where team members attempt to fingerprint the honeypot, using findings to improve realism.

6.1.2 Challenge T2: State Consistency and Persistence

Description: Maintaining a believable, consistent fake OS state across multiple attacker interactions is complex. The sandbox must ensure that file writes persist, processes remain consistent, logs accumulate appropriately, and bash history reflects actual command execution—all without contradictions that reveal the deception.

Impact: High - Inconsistent state immediately breaks believability

Solutions Implemented:

- **Transactional State Updates:** Use SQLite transactions to ensure atomic state updates. If any part of a state mutation fails, rollback the entire operation to maintain consistency.
- **State Validation Rules:** Implement validation logic that checks for logical consistency before committing state changes. For example, verify parent directories exist before creating files, ensure PIDs are unique, check disk space limits.
- **Comprehensive Unit Tests:** Develop extensive test suites that verify state behavior under various scenarios, including edge cases and error conditions.
- **State Snapshot Mechanism:** Create periodic snapshots that allow rollback to known-good states if corruption is detected.
- **Logging and Auditing:** Maintain detailed audit logs of all state mutations to diagnose inconsistencies and improve the state management logic.

6.1.3 Challenge T3: Component Integration Complexity

Description: Integrating five distinct components (SSH Frontend, HTTP Frontend, AI Engine, Sandbox Store, Dashboard) with different technologies and communication patterns creates

coordination challenges. Version mismatches, API contract violations, and asynchronous communication issues can cause system failures.

Impact: High - System cannot function without proper integration

Solutions Implemented:

- **API-First Development:** Define comprehensive API specifications using OpenAPI (Swagger) before implementation. All components must adhere to these contracts with automated validation.
- **Mock Services:** Create mock implementations of each component early in development, allowing parallel work without blocking dependencies.
- **Integration Testing from Week 1:** Establish continuous integration pipeline that runs integration tests on every commit, catching interface mismatches immediately.
- **Weekly Integration Sprints:** Dedicate specific time each week for all team members to work together on integration issues, ensuring regular synchronization.
- **Communication Standards:** Use JSON for all data exchange with versioned schemas. Implement backward compatibility to handle version mismatches gracefully.

6.1.4 Challenge T4: Performance and Response Latency

Description: AI inference, especially with transformer models, can introduce significant latency (2-5 seconds on CPU). If responses are too fast, they appear scripted; if too slow, they become suspicious or frustrating for attackers who expect realistic system performance.

Impact: Medium - Affects believability and attacker engagement

Solutions Implemented:

- **Early Profiling:** Profile AI inference performance from week 5, identifying bottlenecks and optimization opportunities before they become critical.
- **Response Caching:** Cache AI responses for common commands (ls, pwd, whoami) with appropriate variations. Use cached responses to reduce inference calls by 60-70%.
- **Model Optimization:** Employ model quantization and pruning techniques to reduce inference time. Consider using DistilBERT or other lightweight models if latency is problematic.

- **Asynchronous Processing:** Implement async FastAPI endpoints with background tasks for AI inference, preventing blocking and improving throughput.
- **Strategic Delay Insertion:** Add artificial delays to fast responses to simulate realistic command execution time, making response timing appear natural rather than instantaneous or uniformly slow.

6.1.5 Challenge T5: Security Vulnerabilities and Host System Protection

Description: The honeypot must be immune to exploitation by attackers. Any vulnerability that allows command execution on the host OS, container escape, or unauthorized access to Dashboard would be catastrophic. Input validation, isolation, and secure coding practices are critical.

Impact: Critical - System security breach could compromise project and deployment environment

Solutions Implemented:

- **Zero Real Command Execution:** Fundamental design principle—no attacker input ever reaches real shell execution. All commands processed only in sandbox simulation.
- **Container Isolation:** Deploy all components in Docker containers with minimal privileges (non-root users), read-only filesystems where possible, and restricted network access.
- **Input Sanitization:** Implement strict input validation for all attacker-controlled data. Use allowlists for command parsing and escape all special characters before any processing.
- **Security Code Reviews:** Conduct peer reviews focused on security implications for all code handling external inputs. Use automated security scanning tools (Bandit, Safety).
- **Library Selection:** Use well-established, actively maintained libraries (Paramiko, AsyncSSH) with strong security track records rather than custom implementations.

6.2 Organizational and Team Challenges

6.2.1 Challenge O1: Conflicting Academic Schedules

Description: Team members have different course loads, exam schedules, and personal commitments throughout the semester. Ensuring consistent availability for meetings, integration work, and milestone delivery can be difficult.

Impact: Medium - Can cause delays and coordination issues

Solutions Implemented:

- **Flexible Timeline with Buffers:** Build 2-3 week buffer time into the 25-week schedule to accommodate unexpected delays.
- **Clear Task Ownership:** Assign primary responsibility for each component to specific team members, reducing interdependencies and allowing independent progress.
- **Cross-Training:** Ensure all team members understand the overall system architecture and can provide backup for critical tasks.
- **Asynchronous Communication:** Use Discord/Slack for continuous updates, reducing dependency on synchronous meetings. Document all decisions in shared documents.
- **Early Milestone Front-Loading:** Complete high-risk and critical-path items early in the semester when schedules are less constrained.

6.2.2 Challenge O2: Limited GPU/Compute Resources

Description: Training and running transformer models requires significant computational resources. Team members may lack access to high-end GPUs, and cloud computing costs may exceed budget constraints.

Impact: Medium - Can limit model complexity and training iterations

Solutions Implemented:

- **University Resources:** Leverage university GPU clusters and lab computers for model training and testing.
- **Free Cloud Tiers:** Utilize Google Colab Pro, Azure for Students, or AWS Educate credits for GPU access.

- **Lightweight Models:** Select efficient models (DistilBERT, BERT-tiny) that can run on CPU with acceptable performance.
- **Response Caching:** Implement aggressive caching to minimize inference calls, reducing compute requirements.
- **CPU Optimization:** Optimize inference pipeline for CPU execution using quantization and ONNX runtime acceleration.

6.2.3 Challenge O3: Insufficient Real Attacker Data

Description: Training and evaluating the AI model requires realistic attacker interaction data. Publicly available datasets may be limited, outdated, or not representative of current attack patterns.

Impact: Medium - Affects model training quality and evaluation validity

Solutions Implemented:

- **Public Dataset Utilization:** Use Cowrie logs, HoneyDB data, and research datasets from academic publications.
- **Synthetic Data Generation:** Create synthetic attack scenarios based on MITRE ATT&CK techniques and documented APT campaigns.
- **Early Deployment:** Deploy a basic scripted honeypot early in the project to start collecting real attacker interactions.
- **CTF and Simulation:** Use Capture The Flag exercises and internal attack simulations to generate training data.
- **Community Collaboration:** Reach out to cybersecurity research community for data sharing opportunities (with appropriate ethical approvals).

6.3 Mitigation Strategy Summary

The project employs a multi-layered risk mitigation approach:

1. **Proactive Planning:** Identify risks early and plan mitigation strategies before problems occur

2. **Iterative Development:** Use agile methodology with frequent integration to catch issues early
3. **Redundancy:** Build backup solutions (cached responses, fallback logic) for critical failure points
4. **Continuous Testing:** Maintain comprehensive test suites and CI/CD pipeline
5. **Regular Communication:** Weekly team meetings and supervisor check-ins to address emerging issues
6. **Documentation:** Thorough documentation enables knowledge transfer and reduces bus factor risk

7 Work Summary (Fall Semester)

7.1 Research Completed

7.1.1 Academic Literature Review

The team conducted comprehensive analysis of academic research in AI-driven honeypot systems and deception technologies. Key papers reviewed include:

- Gaddam (2025): AI-Enhanced Honeypots for Advanced Cyber Deception Strategies - provided empirical validation for adaptive honeypot effectiveness
- Kareem, Sachan, and Malviya (2024): AI-Driven Adaptive Honeypots for Dynamic Cyber Threats - informed our approach to state persistence and reinforcement learning optimization
- Multiple papers on MITRE ATT&CK integration and threat intelligence frameworks

Findings from this research directly influenced our system design decisions, particularly regarding AI model selection, state management requirements, and intelligence extraction mechanisms.

7.1.2 Market and Technology Survey

Extensive analysis of existing honeypot solutions was conducted, including hands-on testing and deployment of:

- **Cowrie:** Tested SSH interaction capabilities and logging mechanisms. Identified limitations in response variety and state persistence that our system aims to overcome.
- **Dionaea:** Evaluated multi-protocol support and malware capture. Recognized the value of protocol diversity while noting the low-interaction limitations.
- **OpenCanary:** Examined alerting mechanisms and deployment simplicity. Understood the tradeoff between ease of use and intelligence depth.
- **T-Pot:** Analyzed integrated platform approach and visualization capabilities. Informed our Dashboard design decisions.

This comparative analysis validated our gap identification and confirmed the need for an adaptive, AI-driven solution that combines high-interaction capabilities with practical deployment requirements.

7.1.3 Technology Stack Research

Team members investigated and evaluated various technologies for each system component:

- **AI/ML Frameworks:** Compared PyTorch, TensorFlow, and HuggingFace Transformers. Selected HuggingFace for ease of use and model availability.
- **Web Frameworks:** Evaluated Django, Flask, and FastAPI. Chose FastAPI for performance, async support, and automatic API documentation.
- **Frontend Technologies:** Reviewed React, Vue, and Angular. Selected React with TypeScript for component reusability and type safety.
- **SSH Libraries:** Tested Paramiko, AsyncSSH, and custom implementations. Chose AsyncSSH for asynchronous capabilities.

7.2 Design Artifacts Produced

7.2.1 System Architecture Diagrams

The team developed comprehensive architecture diagrams illustrating:

- Overall system architecture with component interactions (Figure 1)
- Data flow diagram showing information movement between components (included in Component Breakdown section)
- Network architecture showing isolation boundaries
- Deployment architecture with Docker container structure

7.2.2 API Specifications

Detailed API contracts were defined for inter-component communication:

- Frontend to AI Engine API (command submission, response retrieval)
- AI Engine to Sandbox Store API (state read/write operations)
- Component to Dashboard Backend API (telemetry forwarding)
- Dashboard Backend REST API (session queries, IOC retrieval, metrics)
- WebSocket protocol for real-time updates

7.2.3 Database Schema Design

Complete database schema for Sandbox State Store including:

- Filesystem table with path, content, permissions, timestamps
- Process table with PID, name, user, status, resource usage
- Log table with timestamp, source, severity, message
- History table with session-linked command sequences
- Environment variables table with scope management

7.2.4 Dashboard Mockups

Visual mockups were created for all Dashboard views:

- Live Sessions view with real-time attacker activity
- Session Playback interface with timeline controls
- IOC Management view with filtering and enrichment
- ATT&CK Heatmap visualization
- Analytics and Metrics comparison charts

7.3 Implemented and Prototyped Features

7.3.1 Prototype Components

Basic prototypes were developed to validate key technical approaches:

SSH Frontend Prototype:

- Minimal AsyncSSH server accepting connections
- Basic authentication handler (accept all credentials for testing)
- Command echo functionality
- Session logging to console

AI Response Generator Proof-of-Concept:

- Simple HuggingFace model integration
- Basic prompt construction with command input
- Response generation for common commands
- Timing measurement for inference latency

Sandbox State Store Prototype:

- SQLite database initialization
- Basic filesystem table with CRUD operations

- Simple query API for state retrieval
- Example state snapshot functionality

7.3.2 Testing and Validation

Initial testing was conducted to validate design decisions:

- Deployed existing honeypots (Cowrie) and collected attacker logs for analysis
- Tested AI model inference latency on available hardware
- Validated SQLite performance for expected workload
- Prototyped WebSocket real-time communication

7.4 Individual Team Member Contributions

7.4.1 Ali Ahmed Reda (202201006) - AI Engine and IOC Extraction

Fall Semester Contributions:

- Researched transformer models and selected appropriate architectures for the project
- Developed AI response generation proof-of-concept
- Designed prompt construction methodology for context-aware responses
- Investigated IOC extraction techniques using NER models
- Created initial MITRE ATT&CK mapping logic
- Contributed to overall system architecture design
- Prepared AI/ML sections of project documentation

Key Deliverables:

- AI Engine component specification
- Prototype response generator
- IOC extraction module design document

7.4.2 Ali Nazeer (202100732) - SSH/HTTP Frontends and Dashboard

Fall Semester Contributions:

- Implemented SSH frontend prototype using AsyncSSH
- Researched HTTP honeypot techniques and scanner detection
- Designed Dashboard mockups and user interface flows
- Developed initial Dashboard architecture and technology stack plan
- Set up development environment and Docker configuration
- Created project GitHub repository and version control workflow
- Contributed to system integration design

Key Deliverables:

- SSH frontend prototype
- Dashboard mockups and specifications
- DevOps and deployment documentation

7.4.3 Ahmed Yasser (202201883) - Sandbox State Store

Fall Semester Contributions:

- Designed complete database schema for sandbox state
- Implemented Sandbox State Store prototype with SQLite
- Researched state persistence and consistency mechanisms
- Developed validation logic for state mutations
- Created state snapshot and rollback mechanisms
- Tested existing honeypots (Cowrie, Dionaea) for comparison
- Contributed to data flow and architecture documentation

Key Deliverables:

- Database schema design
- Sandbox State Store prototype
- State management API specification

7.4.4 Abdulkhaliq Sarwat (202202084) - HTTP Frontend and Integration

Fall Semester Contributions:

- Researched HTTP/web honeypot techniques
- Designed HTTP frontend architecture with FastAPI
- Investigated scanner detection and payload analysis methods
- Developed API contract specifications for component communication
- Created integration testing strategy
- Tested existing web honeypots for feature comparison
- Contributed to system design and work breakdown structure

Key Deliverables:

- HTTP frontend design specification
- API contract documentation
- Integration testing plan

7.5 Documentation and Reports

The team produced the following documentation during Fall semester:

- **Project Proposal:** Initial problem statement, solution overview, and project plan
- **Initial System Design Report:** Architecture diagrams, component breakdown, data flow, and work breakdown structure

- **This Final Report:** Comprehensive documentation covering all aspects of the project including literature review, requirements analysis, detailed system design, timeline, challenges, and work summary
- **GitHub Repository:** Code prototypes, documentation, and project wiki

7.6 Supervisor Feedback Integration

Throughout the fall semester, the team met regularly with supervisor Dr. Ashraf Hafez Badawi. Key feedback received and addressed:

- **Scope Refinement:** Initially proposed features were too ambitious for one-semester implementation. Scope was refined to focus on SSH/HTTP protocols and essential Dashboard features.
- **Security Emphasis:** Supervisor emphasized the critical importance of zero real command execution and container isolation. Design was adjusted to prioritize security.
- **Evaluation Methodology:** Feedback received on evaluation plan led to incorporation of baseline honeypot comparison and defined success metrics.
- **Documentation Standards:** Guidance on documentation structure and SO mapping ensured compliance with graduation project requirements.

7.7 Lessons Learned

Technical Insights:

- AI model selection requires careful balance between capability and resource requirements
- State consistency is more complex than initially anticipated
- Early prototyping is essential for validating technical feasibility
- Existing honeypot solutions provide valuable reference but have significant limitations

Project Management Insights:

- Clear component ownership reduces coordination overhead

- Regular integration points prevent late-stage integration crises
- Buffer time in schedule is essential for risk mitigation
- Documentation should be created concurrently with design, not retrospectively

Team Collaboration Insights:

- Asynchronous communication tools (Discord) enable flexible collaboration
- Weekly team meetings maintain alignment and motivation
- Cross-training on components improves overall understanding
- Peer review of work products improves quality

References

[1] N. Gaddam, *AI-Enhanced Honeypots for Advanced Cyber Deception Strategies*, QITP-IJCSRD: International Journal of Cyber Security Research and Development, vol. 5, no. 1, pp. 9–19, Jan.–Jun. 2025.

[2] S. A. Kareem, R. C. Sachan, and R. K. Malviya, *AI-Driven Adaptive Honeypots for Dynamic Cyber Threats*, SSRN Electronic Journal, Sept. 17, 2024.

[3] HuggingFace Team, *Transformers: State-of-the-art Natural Language Processing*, https://huggingface.co/docs/transformers/index, accessed December 2025.

[4] MITRE Corporation, *MITRE ATT&CK Framework*, https://attack.mitre.org/, accessed December 2025.

[5] M. Oosterhof, *Cowrie SSH/Telnet Honeypot*, https://github.com/cowrie/cowrie, accessed October 2025.

[6] S. Ramírez, *FastAPI: Modern, fast (high-performance), web framework for building APIs*, https://fastapi.tiangolo.com/, accessed October 2025.

[7] R. Kern, *AsyncSSH: Asynchronous SSH for Python*, https://asyncssh.readthedocs.io/, accessed November 2025.

[8] Docker Inc., *Docker Documentation*, https://docs.docker.com/, accessed November 2025.

[9] Meta Platforms Inc., *React: A JavaScript library for building user interfaces*, https://react.dev/, accessed November 2025.

A Acronyms and Abbreviations

Acronym	Definition
AI	Artificial Intelligence
API	Application Programming Interface
APT	Advanced Persistent Threat
ATT&CK	Adversarial Tactics, Techniques & Common Knowledge
BERT	Bidirectional Encoder Representations from Transformers
CERT	Computer Emergency Response Team
CI/CD	Continuous Integration / Continuous Deployment
CTF	Capture The Flag
HTTP	Hypertext Transfer Protocol
HTTPS	HTTP Secure
IOC	Indicator of Compromise
JSON	JavaScript Object Notation
ML	Machine Learning
NER	Named Entity Recognition
REST	Representational State Transfer
SIEM	Security Information and Event Management
SME	Small and Medium Enterprise
SOC	Security Operations Center
SSH	Secure Shell
STIX	Structured Threat Information Expression
TAXII	Trusted Automated Exchange of Intelligence Information
TTP	Tactics, Techniques, and Procedures
TTY	Teletypewriter (terminal)